RESEARCH PROJECT REPORT

#68-Assessing the Suitability of Blockchain Frameworks for New Zealand General Elections[1]

Eason Chang

Co-worker: Patrick Sheng

Supervisor: Dr Dev Singh

Co-supervisor: Prof Kevin Sowerby

Department of Electrical, Computer, and Software Engineering

The University of Auckland

16 October 2026

[1]The title of this report has been modified from the original "Assessing the Performance and Scalability of MACI as a Standalone DLT Framework", due to a narrowed scope.

ABSTRACT

New Zealand general elections are still run on paper, and any online alternative must count each eligible vote exactly once, keep the result auditable and each vote secret, and stay available throughout voting. Earlier work found that a public blockchain approach, MACI, could keep votes secret but was too slow at national scale. This report evaluates the infrastructure of an alternative: a private, permissioned blockchain built on Hyperledger Besu with the IBFT 2.0 consensus protocol, in which a known group of validators shares the election record and a voting contract holds the voter roll and the tally. Two tests were run on a four-validator network. In a tally audit, every validator held an identical record in all six runs, including after a validator crashed and recovered, and the stored tally matched an independent recount of 10,000 votes; scaled to 3.7 million votes, a recount would take about 12 minutes per validator. In a storage test, each vote added about 3.5 kB, giving an estimated 19.5 GB per validator for a national election. Capacity and availability are drawn from a companion report. The results suggest that a private blockchain can give several organisations a shared, independently checkable election record. However, they were obtained at small scale on a single machine, the prototype stores every vote openly and so provides no ballot secrecy, and ballot integrity depends on a single administrator key and an identity check outside the chain. A private, permissioned blockchain is therefore a potentially viable foundation for online voting, but not a complete solution, and its suitability depends on further design and evaluation.

SIGNED STATEMENT OF CONTRIBUTION

Student

I hereby declare that:

1. This report is the result of the final year project work carried out by my project partner (see cover page) and I under the guidance of our supervisor (see cover page) in the 2026 academic year at the Department of Electrical, Computer, and Software Engineering, Faculty of Engineering and Design, University of Auckland.
2. This report is not the outcome of work done previously.
3. This report is not the outcome of work done in collaboration, except that with a potential project sponsor (if any) as stated in the text.
4. This report is not the same as any report, thesis, conference article or journal paper, or any other publication or unpublished work in any format.

In the case of a continuing project, please state clearly what has been developed during the project and what was available from previous year(s):

The 2025 project benchmarked e-voting on public Ethereum infrastructure, and its results were used here as a baseline only. This year we designed and built a private permissioned network using Hyperledger Besu with IBFT 2.0 consensus, along with the voting smart contract, the frontend, and all benchmarking and analysis.

Signature: 

Date: 17/10/2026

Supervisor

I confirm that the project work undertaken by this student in the 2026 academic year is / is ~~not~~ (strikethrough as appropriate) part of a continuing project, components of which have been completed previously. Comments, if any:

Signature: 

Date: 16/10/2026

Table of Contents

Acknowledgements . . . . . **vi**

Glossary of Terms . . . . . **vii**

Abbreviations . . . . . **ix**

1 Introduction **1**

2 Literature Review **1**
2.1 Requirements for Trustworthy National Voting 1
2.2 Online Voting and the Problem of Trust 3
2.3 Distributed Ledger and Blockchain Fundamentals 3
2.4 Public and Private Blockchains 5
2.4.1 Public Blockchains and MACI 5
2.4.2 Private Blockchains 5
2.5 Byzantine Fault Tolerance and IBFT 2.0 6
2.6 Performance Studies of Private Blockchain Platforms 6
2.7 Research Gap and Aim 7

3 System Design and Implementation **7**
3.1 System Overview 8
3.2 The Validator Network on Besu 8
3.2.1 Nodes and Peer Connections 9
3.2.2 Genesis File and Network Size 10
3.2.3 Permissions and the Administrator Role 11
3.3 The Voting Contract 11
3.4 Vote Validation 12
3.4.1 Transactions and the Transaction Pool 12
3.4.2 Checking Eligibility Against the Voter Roll 13
3.4.3 Consensus 13
3.5 Storage 13
3.5.1 Blocks 13
3.5.2 Receipts and Events 14
3.5.3 World State 14
3.6 Tallying 15

4 Evaluation Method **16**
4.1 Test Environment 16
4.2 Election Load Scenarios 16
4.3 Tally Audit Across Validators 16
4.4 Storage Growth 17
4.5 Threats to Validity 17

5 Results **18**
5.1 Tally Audit Across Validators 18
5.2 Storage Growth 19

6 Discussion **20**
6.1 Tally Integrity and Audit 20
6.1.1 Consistency Across Validators 20

6.1.2 Recount Time at National Scale 20
6.1.3 What the Audit Cannot Catch 20
6.2 Storage at National Scale 21
6.3 Capacity and Availability 22
6.4 Voter Privacy and Ballot Secrecy 22
6.5 Comparison with MACI 23
6.6 Practical Deployment 23
6.7 Suitability Against the Criteria 24

7 Conclusions and Future Work 25

References 27

Appendix A Example Block and Receipt 30

Acknowledgements

I would like to thank my supervisor, Dr Dev Singh, and co-supervisor, Prof Kevin Sowerby, for meeting with us every week throughout the year. Their guidance shaped the direction and content of this project, and their advice went well beyond the project itself, offering lessons I will carry with me.

I am grateful to my project partner, Patrick Sheng, for his hard work and for working through many of the project's difficulties alongside me. I also thank Chris Kwon and Joshua Browne, whose evaluation of MACI in 2025 laid the groundwork this project builds on.

Thank you to everyone in the Radio Lab, who shared each day with us and were always willing to listen to our project and discuss it with us. Finally, thank you to my family and friends for their support throughout the year.

Glossary of Terms

Advance vote A vote cast before election day.

Auditability The ability to detect any fault that changes the result, and to check the outcome from the records kept.

Availability The ability of a voting system to remain usable for every voter throughout the voting period, including when parts of it fail or are attacked.

Ballot integrity The principle of one person, one vote: each eligible voter has exactly one vote counted.

Ballot secrecy The guarantee that no record links a voter's identity to their choices.

Block A batch of transactions added to a blockchain, which stores the hash of the previous block.

Blockchain A distributed ledger that groups transactions into blocks linked in sequence by hashes.

Byzantine fault A failure in which a node behaves arbitrarily, including crashing or sending wrong or conflicting messages.

Consensus protocol The rules by which nodes agree on which transactions enter the ledger and in what order.

Finality The point at which a committed block can no longer be reversed. Immediate finality means this happens as soon as the block is committed.

Gas A unit measuring the computing work a transaction needs. A sender pays a fee for each unit at the gas price, which is zero in this project.

Hash A short, fixed-length value computed from data, which changes completely if the data changes.

Ledger An ordered record of transactions.

Node A computer that runs blockchain software and holds a copy of the ledger.

Ordinary vote A vote cast at a voting place by a voter whose name is found on the roll for that electorate.

Private blockchain A blockchain in which only approved parties may read the ledger and validate blocks. Limiting validation to approved validators makes it permissioned.

Public blockchain A blockchain that anyone may read and help validate. Allowing anyone to validate makes it permissionless.

Quorum The minimum number of validators that must agree before a block is committed.

Scrutineer A person appointed by a candidate or party to observe voting and counting.

Smart contract A program stored on a blockchain that every node executes in the same way.

Special vote A vote checked for the voter's eligibility before it is counted, for example

from a voter who is overseas or not yet on the roll.

Transaction pool The queue in which a node holds submitted transactions until they are included in a block.

Transparency Openness of a voting system's documentation, processes and transactions to inspection.

Validator A node approved to propose and approve blocks in a permissioned blockchain.

Voter privacy The ability of each voter to mark, check and cast a ballot alone and without help.

Zero-knowledge proof A cryptographic proof that a statement is true without revealing the underlying data.

Abbreviations

BFT Byzantine Fault Tolerance

EVM Ethereum Virtual Machine

IBFT Istanbul Byzantine Fault Tolerance

JSON-RPC JavaScript Object Notation Remote Procedure Call

MACI Minimal Anti-Collusion Infrastructure

PBFT Practical Byzantine Fault Tolerance

QBFT Quorum Byzantine Fault Tolerance

VVSG Voluntary Voting System Guidelines

zk-SNARK Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

1. Introduction

New Zealand general elections are still run on paper. In 2023, 2,884,111 votes were cast, and every ordinary vote was counted by hand twice before the official result was declared 20 days after election day [1], [2]. Online voting could make voting easier for people who struggle to reach a voting place and could produce results faster, but a national election places strict demands on any voting system: every eligible vote must be counted exactly once, the result must be checkable, each voter's choice must stay secret, and the system must stay available for the whole voting period. A planned online voting trial for the 2016 local body elections was cancelled before it ran [3], and the underlying difficulty remains trust, because a conventional online system asks voters to trust whoever runs its servers. Blockchain technology has been proposed as one way to spread that trust across several independent parties, although it is one candidate approach among several rather than the only path. The 2025 Part IV project that preceded this one evaluated Minimal Anti-Collusion Infrastructure (MACI), which runs on public blockchains and uses zero-knowledge proofs to keep votes private [4], [5]. It found that these guarantees are costly: vote confirmation on public networks was slow, and producing the proofs needed to verify a national tally was estimated to take weeks. Kwon [5] suggested that a private blockchain run by known organisations should scale better, but this option was not tested.

The project this report belongs to asks whether blockchain frameworks are suitable for a New Zealand general election, and tests one candidate: a private network built on Hyperledger Besu with the IBFT 2.0 consensus protocol. This report focuses on that network's infrastructure, meaning how its nodes are set up, how they communicate, and how the vote record is stored and tallied, and evaluates it against the requirements of a New Zealand general election. The voting smart contract and the consensus protocol are examined in more depth by my project partner, P. Sheng [42]. Section 2 reviews the requirements and the relevant literature, and ends with the aim of this evaluation and the areas it covers. Section 3 describes the design and implementation of the network, Section 4 the evaluation method, and Section 5 the results. Section 6 interprets the results, and Section 7 concludes.

2. Literature Review

Ground already covered in the 2025 MACI reports, such as the benefits and risks of online voting, is summarised briefly with reference to those reports. More depth is given to private blockchains, Byzantine fault-tolerant consensus and performance studies of private blockchain platforms, which those reports did not examine.

2.1 Requirements for Trustworthy National Voting

Before a voting system can be assessed, the requirements it must meet need to be stated explicitly, and the literature offers no single agreed list. Browne [4] took four principles from the United States Election Assistance Commission's Voluntary Voting System Guidelines 2.0 (VVSG 2.0) and added a fifth from Loncke and Dumortier [6], while Kwon [5] grouped the same guidelines into four different categories. This report adopts Browne's five criteria and adds a sixth. VVSG 2.0 has one important limitation here: it was written for in-person voting equipment, and it does not allow any device with an external network connection to form part of a certified voting system [7]. It is used here as a precise vocabulary of requirements, not as a standard an online system could be certified against.

The six criteria are:

- ***Transparency***: a system's documentation, processes and transactions are open to inspection, so the public can understand and check how the election runs [7].
- ***Voter privacy***: each voter can mark, check and cast a ballot alone and without help [7].
- ***Ballot secrecy***: narrower than voter privacy, it requires that no record the system produces may link a voter's identity to their choices [7].
- ***Auditability***: a fault cannot change the result without being detected, and the system keeps records from which the outcome can be checked [7].
- ***Ballot integrity***: the principle of one person, one vote, which requires that each eligible voter has exactly one vote counted [6].
- ***Availability*** (added by this report): the system remains usable for every voter throughout the voting period, including when parts of it fail or are attacked.

On availability, VVSG 2.0 already asks that voting systems recover gracefully from failure and that no single point of failure can halt voting at a polling place [7]. For an online system, the risk is larger because a denial-of-service attack can block many voters at once and can be aimed at particular groups [4], [5], [36], [37]. These criteria are not independent. Transparency and auditability favour publishing as much of the election record as possible, while ballot secrecy requires that the link between a voter and their choice is never recorded at all. Any digital design must decide how to resolve this tension, and that choice shapes the rest of this review.

New Zealand's paper system is the baseline that any alternative must at least match. Ballots are marked in private, and secrecy provisions prevent anyone from requesting to see a person's voting papers. Scrutineers appointed by candidates and parties may observe the issuing of votes and every count, ordinary votes are counted twice by hand, results are published for each voting place, and recounts are supervised by a judge [1]. Ballot integrity relies on each voter being marked off the electoral roll when a ballot is issued [43], and on special votes being checked for eligibility before they are counted [8]. More than 2,300 voting places were open on election day in 2023, so the failure of any one site affects only the voters near it [9]. The weakness of this model is that its assurance is indirect. A voter cannot confirm that their own ballot was counted, and must instead trust that officials and observers have done their jobs [4].

The paper system also sets the scale and timing a replacement would face. About 1.39 million ordinary votes were cast in advance and about 0.9 million on election day itself [2], which is an average of roughly 25 votes per second over the ten hours voting places were open [9]. Because hourly figures are not published, this can only be stated as a daily average, and the real peak would be higher. The 2025 MACI reports used rates of 6.57 [5] and just over seven [4] votes per second. These rates match the 260,219 votes recorded on 13 October, the last day of advance voting [10], and so understate election-day demand by a factor of nearly four. Preliminary results are released on election night and official results 20 days later [1]. Browne [4] used these timings to set two targets for an online system: a tally within one hour of voting closing, and evidence that the tally is correct within one day. The paper system therefore meets the criteria largely through people and

physical procedure, at the cost of in-person attendance and a final count that takes weeks. These are the costs that online voting promises to reduce.

2.2 Online Voting and the Problem of Trust

Online voting offers clear practical benefits. Voters can take part from anywhere, people with mobility or vision impairments may be able to vote without help, and electronic counting can be faster than counting by hand [4], [5]. The evidence that it raises turnout is weaker. Estonia is the most established example of national internet voting, having offered it since 2005 [38], and in its 2023 parliamentary election just over half of all votes were cast online [11]. Yet a study of eleven Estonian elections found that, although internet voting was widely used and trusted, it did not raise turnout in a setting where voting was already easy [12].

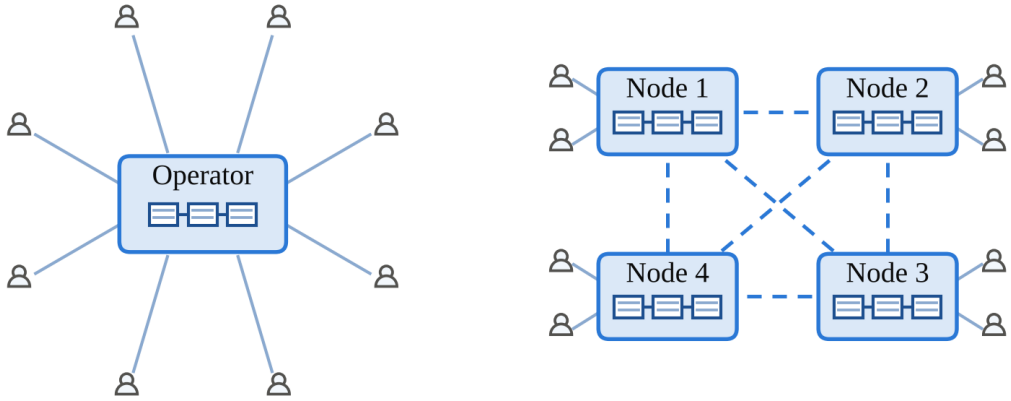
The main risk relates to trust. In a centralised system such as Estonia's, votes are collected and counted on servers run by the election authority, so voters must trust that operator and its procedures. Springall et al. [13] reviewed the Estonian system and, working on a reproduction of it, demonstrated attacks on both the election servers and voters' computers that could alter results or undermine confidence in them; Browne [4] notes that the Estonian government disputed these findings. Park et al. [14] go further, arguing that remote online voting makes large-scale, undetectable failures more likely, because a single software flaw or compromised device can affect many votes without leaving the physical evidence a paper ballot provides. New Zealand's own planned trial reflects these concerns. The Department of Internal Affairs cancelled it because it was not satisfied that there was enough time to develop and test the trial, and several councils had already withdrawn because of cost or security concerns [3].

These risks are not all of the same kind. Some, such as malware on voters' devices, lie outside any server-side design and are outside the scope of this project. Others arise because a single organisation holds the only authoritative copy of the votes and the software that counts them. This second problem is the one that distributed ledgers claim to address: instead of trusting one operator, can trust be spread across several independent parties who must agree?

2.3 Distributed Ledger and Blockchain Fundamentals

A ledger is an ordered record of transactions; in a voting system, each vote is a transaction. In a centralised ledger, such as a conventional database, one organisation holds the authoritative copy and decides what is written to it, so anyone relying on the record must trust that organisation not to alter it and to keep it available. A distributed ledger instead keeps a full copy of the record on many computers, called nodes, which follow shared rules to agree on its contents (Figure 1).

A blockchain is a distributed ledger that groups transactions into blocks and links the blocks in sequence [15]. The links between blocks are made with cryptographic hashes. A hash function turns any data into a short, fixed-length value, and even a one-character change to the data produces a completely different value. Each block stores the hash of the block before it, so altering a vote in an old block changes that block's hash, which then no longer matches the value stored in the next block [15]. This makes tampering evident rather than impossible, since a single node could still rewrite its own copy. What protects the record is the combination of hashing with many independent copies: a rewritten copy would disagree with all of the others.



(a) Centralised ledger (b) Distributed ledger

Figure 1: Centralised and distributed ledgers. (a) One operator holds the only copy of the ledger. (b) Every node holds a full copy, and the nodes agree on each change.

Because every node holds a copy, the nodes need a way to agree on which transactions enter the ledger and in what order, even if some nodes fail or behave dishonestly. This is the job of a consensus protocol. Open networks such as Bitcoin, which anyone can join, rely on mechanisms such as proof of work that make it costly to outvote honest nodes [15]. Networks whose members are known can use voting-based protocols instead, which Section 2.5 explains. Some blockchains, such as Ethereum, also run smart contracts: programs stored on the ledger that every node executes in the same way, so that all nodes reach the same result [39].

Against the criteria in Section 2.1, these properties fit some requirements well and conflict with others, listed below from greatest benefit to greatest conflict:

- *Ballot integrity and auditability* benefit most: every node holds the full record, the rules are applied identically everywhere, and the tally can be recomputed from the ledger by anyone with read access.
- *Availability* can improve, since the record has no single point of failure, although consensus itself needs enough nodes to be running (Section 2.5).
- *Transparency* depends on who is allowed to read the ledger.
- *Ballot secrecy* is where the conflict lies. A ledger that is copied to many parties and kept permanently is the opposite of a sealed ballot box: if a vote can be linked to a voter, that link is replicated to every node and cannot easily be removed [5], [14]. This report does not resolve this conflict, and Section 2.7 explains why it lies outside the evaluation.

Park et al. [14] also point out that a blockchain is essentially a shared bulletin board. It does not protect a vote on a compromised device before it reaches the ledger, and it does not by itself give voters evidence that their vote was recorded as they intended. A blockchain therefore moves trust from one operator to agreement among several parties, rather than removing it. Whether the blockchain is public or private determines who these parties are.

2.4 Public and Private Blockchains

2.4.1 Public Blockchains and MACI

In a public blockchain, anyone can run a node, read the ledger, and take part in consensus. Ethereum is the best-known public network that runs smart contracts. Openness makes it hard for any single party to control the ledger, but it limits speed because consensus must work among many anonymous participants. MACI was designed for this setting. It encrypts each vote so that only a designated coordinator can read it, lets voters replace an earlier vote so that no one can prove how they voted, and uses zero-knowledge proofs (zk-SNARKs) to show that the coordinator counted correctly without revealing individual votes [4], [5].

The 2025 project measured the cost of these guarantees. Kwon [5] found average vote confirmation times of 10.7 seconds on Ethereum's Sepolia test network, 4.6 seconds on Gnosis Chiado and 1.3 seconds on Optimism Sepolia, a layer 2 network that batches transactions before settling them on Ethereum. Browne [4] found that the coordinator's off-chain processing grew linearly with the number of votes, and estimated that five million votes would take about 2.86 weeks on a single laptop, or about 1.27 weeks with proof generation spread across many machines. Both figures are far beyond the one-day target he set for publishing verified results. Both reports note limits on how far their figures generalise. Kwon's latencies were measured through a free, rate-limited third-party node provider, and Browne's estimates were extrapolated from polls of at most 6,400 votes using a slower proof generator than is available on other hardware. Their conclusion, that MACI on public infrastructure is not yet practical for a national election, rests on bottlenecks large enough that these caveats are unlikely to reverse it. Browne [4] also showed that MACI's openness lets an attacker flood a poll with invalid messages, each of which the coordinator must still process. The slow confirmations and this flooding attack both come from running on an open network shared with other users, which led Kwon [5] to suggest that a private network run by known organisations should perform better.

2.4.2 Private Blockchains

A private blockchain restricts both who may read the ledger and who may validate blocks. Strictly, these are separate choices: reading can be public or private, and validation can be permissionless, open to anyone, or permissioned, limited to approved validators. The two systems compared in this report are the pure cases. Ethereum, where MACI was tested, is public and permissionless, while the network studied here is private and permissioned. This report therefore uses "public" and "private" for these two combinations, and "permissioned" only where the point concerns who may validate. Validators are known organisations approved in advance. Because they are identified, they can reach agreement by exchanging signed messages rather than through costly computation, which allows faster blocks and immediate finality [16] (Section 2.5). In an election, the validators might be the electoral authority and independent bodies such as auditors. Kwon [5] rated private chains as offering very high scalability but only medium security and low decentralisation compared with public networks. That rating is useful as a starting point, but it was derived from documentation of public test networks rather than from measurement, and neither 2025 report tested a private network.

The strongest objection to permissioned voting ledgers comes from Park et al. [14]. They argue that a small set of validators gives an attacker fewer and more similar servers to compromise; that if voters cannot read the ledger they cannot verify their own vote, while

if everyone can read it, voters may be able to prove how they voted; and that elections are inherently centralised, so the decentralisation a blockchain offers does not match how elections are run. These points do not rule out a private design, but they define what it must demonstrate. It must keep working when some validators fail or misbehave, it must give validators and auditors a consistent record they can check independently, and it must be explicit that it does not by itself solve ballot secrecy or device security, which need separate measures: hiding the content of each vote, and securing the devices that voters use. The first two can be measured, and they are the focus of this project.

Whichever platform is used to build it, a private blockchain places trust in a known group of validators, so the next question is how they can agree safely when some of them fail or lie.

2.5 Byzantine Fault Tolerance and IBFT 2.0

A Byzantine fault is any failure in which a node behaves arbitrarily: it may crash and fall silent, or send wrong or conflicting messages to different peers. Castro and Liskov [18] showed with Practical Byzantine Fault Tolerance (PBFT) that up to f such faulty nodes can be tolerated if the network has at least $3f + 1$ nodes in total. Each decision then needs approval from $2f + 1$ of these nodes, a group called a quorum, which is about two-thirds of the network. Because every node exchanges messages with every other node, the number of messages grows with the square of the number of nodes.

With four validators, the validator network tolerates one faulty validator and needs three to agree on each block. Any two groups of three out of four share at least two validators, so at least one honest validator belongs to both and will not approve two conflicting blocks. With only three validators, two groups of two could share a single validator, and if it were faulty both conflicting blocks could be approved, which is why four is the minimum [19]. Seven validators tolerate two faults with a quorum of five.

IBFT 2.0 is a protocol in this family, designed by Saltini and Hyland-Wood [20] to fix flaws in the original Istanbul BFT protocol that could let a single faulty validator reverse a finalised transaction or stop the chain. Two of its properties matter for this report. First, it provides immediate finality: once a block is committed it cannot be reversed, so a voter needs only one confirmation [20]. Second, it puts consistency ahead of availability: if more than one-third of validators stop participating, the network stops producing blocks rather than risk inconsistent records [19]. For an election this is arguably the right trade-off, since a pause can be recovered from but a divergent vote record cannot.

These guarantees cost more as the network grows. Adding validators spreads trust more widely but increases consensus traffic, so blocks take longer to add [16].

2.6 Performance Studies of Private Blockchain Platforms

Several studies have benchmarked private blockchain platforms, but few match the conditions of a national vote. Pongnumkul et al. [22] compared Hyperledger Fabric [17] with a private Ethereum network and found that Fabric performed better in execution time, latency and throughput on the workloads they tested.

Hyperledger Besu, an Ethereum client that offers a private network option and the platform used in this project (Section 3.2) [30], has been studied even less. Mostarda et al. [23] measured a Besu IBFT 2.0 network operated by several organisations at different sites, with a six-second block period and free transactions, using the Hyperledger Caliper benchmarking

tool. Average transaction latency, which Caliper measures as the time from sending a transaction until it is confirmed in a block, rose from about 2.7 seconds to 7.2 seconds as the send rate increased from 0.2 to 10 transactions per second. The authors also found that some validators consistently produced near-empty blocks, so transactions were spread unevenly across blocks. Their network had four validators throughout, so it did not show how performance changes with the number of validators. It also used a single token-transfer workload and repeated each test only twice.

Taken together, this literature leaves two gaps that matter for an election. First, few studies measure how a Besu network's performance changes as validators are added, which is the trade-off identified in Section 2.5. The closest is Tkachuk et al. [44], who increased a Besu network from 4 to 24 validators and found that maximum throughput fell by about 42%, but their workload was an energy-trading marketplace rather than an election. Second, the studies concentrate on throughput and latency under normal operation. They rarely examine what happens to submitted transactions when validators fail, whether all nodes end with the same record, or how storage grows as the record lengthens, yet these are the properties that determine availability and integrity in an election. None of them relates its results to the demand of a real election.

2.7 Research Gap and Aim

The literature reviewed above shows that public-chain approaches such as MACI meet the privacy requirements of Section 2.1 but hit performance bottlenecks at national scale [4], [5], and that private networks are expected to perform better [5], [16]. Existing benchmarks of private platforms, however, measure generic workloads rather than election requirements such as surviving validator failures, keeping an auditable tally or storing a national vote record, and the 2025 project rated private chains without testing them [5]. No evaluation of a private Byzantine fault-tolerant network against the requirements of a New Zealand general election was found in this review.

This report therefore evaluates whether a private Besu network is suitable as the infrastructure for a New Zealand general election. Its own tests examine two areas:

Integrity under failure: whether every validator ends with the same vote record and a tally that can be checked independently, including after a validator crashes and catches up, and how long that check would take for a national election.

Operation at scale: how much storage the record of a national election needs on each validator.

Performance and availability, meaning whether the network processes votes fast enough for New Zealand election demand and how it behaves when validators fail, are examined by P. Sheng [42], and Section 6 draws on those results.

Voter privacy and ballot secrecy are not investigated, because the prototype does not provide them; they are treated as a limitation in the Discussion.

3. System Design and Implementation

This section describes the system built to test whether a private network can keep an election record that no single operator controls: a quorum of known validators must agree on every change, and every validator holds its own copy of the record that can be checked

independently.

3.1 System Overview

Figure 2 places the system in an online election. A voter signs in to a web or app platform, for example through RealMe, the New Zealand government's online identity service, and the platform sends the vote to the voting system, which returns a confirmation once the vote is recorded. The platform and the identity check are outside the scope of this project. In this project, votes were submitted by the test tools described in Section 4.

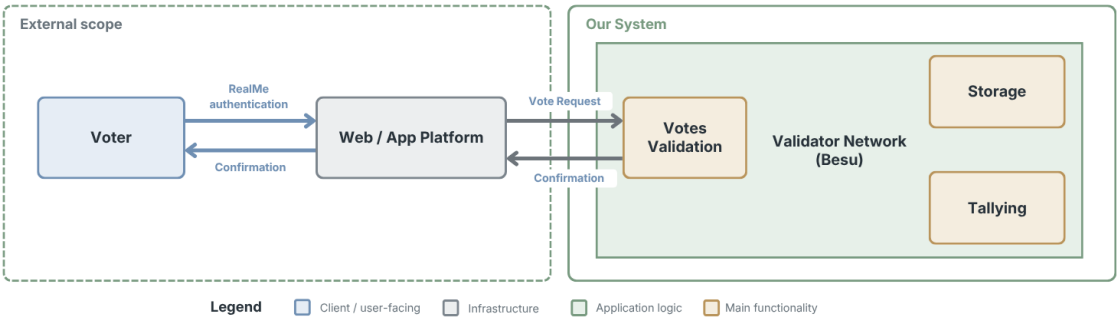


Figure 2: The voting system in the context of an online election. Components inside the dotted box are outside the scope of this project.

This project builds the part labelled Our System. Its work falls into three areas: validating votes, storing the record, and tallying the result. These are not separate components. Every validator does all three, because the blockchain and the voting contract bind them together: the same transaction that validates a vote also stores it and adds it to the tally.

3.2 The Validator Network on Besu

The system is a network of n validators, each a separate Besu node holding its own full copy of the chain. Figure 3 shows what runs inside each one: a transaction pool that holds incoming transactions, IBFT 2.0 consensus that agrees on each block with the other validators, the Ethereum Virtual Machine (EVM) that runs the voting contract, and storage for the blocks, receipts, and world state. Because the election's data and rules live in the voting contract (Section 3.3), there is no separate database or backend server: the election record exists only in the replicated chain. Every validator runs the same parts, so any one of them can accept a vote, check it and answer a query from its own copy.

Figure 3 also shows auditor and observer nodes. These are not validators: they hold a read-only copy of the chain, so an auditor can recount the votes and an observer can confirm that no block has been changed. They lie outside the scope of this project, and the prototype ran validators only.

Each validator runs Hyperledger Besu, an open-source Ethereum client: software that runs a node on Ethereum-compatible networks, both public and private [30]. We chose it for three reasons:

Ethereum compatibility. The voting contract is written in Solidity with standard Ethereum tools, and the results stay comparable with the 2025 MACI work, which ran on Ethereum networks [5].

Support for private networks. Besu provides the permissioning and Byzantine fault-tolerant consensus that a private voting network needs [16], [24].

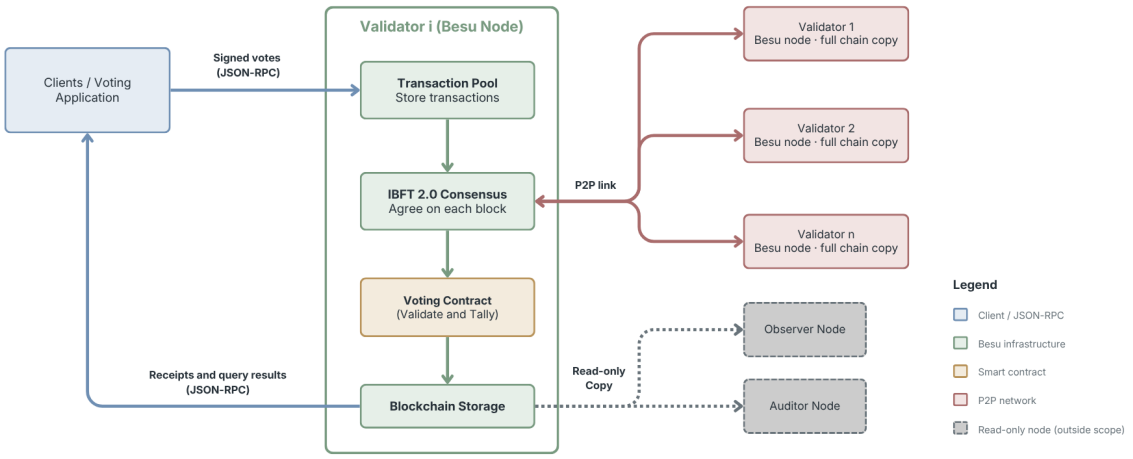


Figure 3: Inside one validator. Every validator runs the same components and holds a full copy of the chain.

Benchmarking support. Besu works with Hyperledger Caliper, the benchmarking tool used in the 2025 project [5].

Using an existing client meant that networking, consensus, and storage did not have to be built from scratch.

Clients reach the network through the JSON-RPC interface that every Besu node exposes, a standard way of sending requests to a node over HTTP. A voter client signs each vote and sends it to any validator, and an administrator client registers voters and opens and closes voting.

3.2.1 Nodes and Peer Connections

Each Besu node has a key pair that serves as its identity on the network. The node's public key, together with the IP address and port at which it can be reached, forms its enode URL [28]. Figure 4 shows an enode URL from the project's first multi-node network. In an IBFT 2.0 network, an address derived from the same public key also identifies the node as a validator [28].

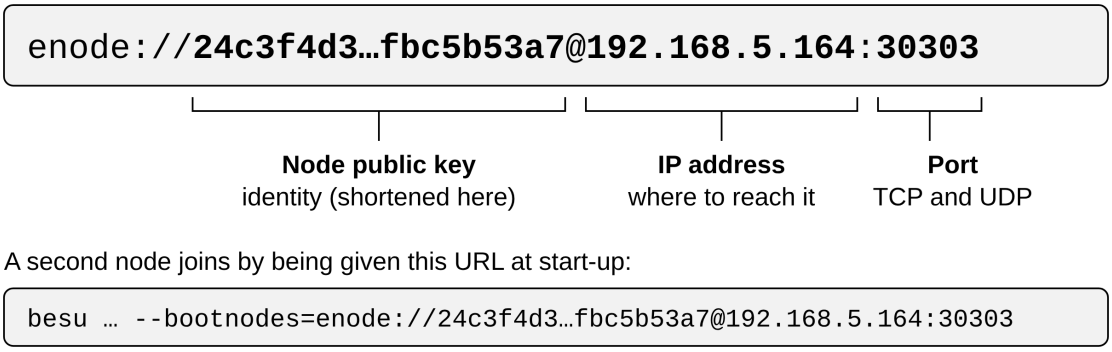


Figure 4: Structure of an enode URL, using a validator from the project's first multi-node network (public key shortened), and how a second node is pointed to it.

Knowing one node's enode URL is not enough to reach the rest of the network. A new node joins the network as follows:

1. At start-up, the node is given the enode URLs of one or more bootnodes [28].

2. Using a UDP discovery protocol, it asks those bootnodes which other nodes they know about [31].
3. It then adds the answers to its own table of peers.
4. It repeats steps 2 and 3 with the newly found peers until it knows enough of them.

In this way, a node can find the whole network while knowing only one address in advance.

Once a node knows a peer, it opens a direct connection to it over TCP. The two nodes begin with an encrypted handshake that also proves each holds the private key matching its identity. Every later message on the connection is encrypted and authenticated [32]. One connection carries several protocols at once [32], so pending transactions, new blocks and IBFT 2.0 consensus messages all travel between two validators over the same link. Each node therefore needs its port open for UDP, used for discovery, and for TCP, used for these connections (Section 3.2.2).

3.2.2 Genesis File and Network Size

The rules of a Besu network are fixed in a genesis file that every node must share. Our genesis file sets the following [19]:

- Consensus protocol: IBFT 2.0
- Chain ID: 1337
- Block period: 2 s
- Round-change timeout: 4 s
- Epoch length: 30,000 blocks
- Initial validators: listed in the file

Each node stores its world state in the Bonsai format (Section 3.5.3). Apart from this, nodes start with Besu's enterprise profile, which in version 24.12.0 selects full synchronisation, the sequenced transaction pool and a minimum gas price of zero [25]. Gas measures the computing work a transaction needs, and the gas price is the fee paid for each unit of it, so a price of zero means voters pay nothing to vote. Because the profile's default storage format is Forest, our start scripts set Bonsai explicitly. The sequenced pool keeps its default capacity of 4,096 transactions (Section 3.4.1).

The number of validators follows the $n = 3f + 1$ rule from Section 2.5. Networks of 4, 7, 10 and 13 validators were built, tolerating one to four faulty validators, so that the trade-off between fault tolerance and speed could be measured.

Forming the network was the main practical task in building the system. The validators were first run on separate machines, with a bootnode on a virtual private server and peers joining from other computers, which showed that they can form a network across the internet. It also showed how strict the setup is: the genesis file must be byte-identical on every node, each node must advertise a reachable enode URL, and port 30303 must be open for both UDP and TCP (Section 3.2.1), or peers fail to connect, often without a clear error. For repeatability, all measurements were then taken on a single host (Section 4).

3.2.3 Permissions and the Administrator Role

Access to the network is controlled at several layers, each answering a different question: which machines may connect, which nodes may create blocks, and which accounts may send transactions [19], [24].

Network. Besu can refuse connections from any node whose enode URL is not on an allowlist [24]. In the intended deployment, each validator would be run by a different trusted organisation, such as the Electoral Commission, an independent auditor or a registrar, and only their nodes would be on the allowlist. An outside machine would then not be able to join the network or exchange messages with the validators at all. Because an enode URL contains the node's public key, the allowlist is tied to each validator's identity, not only to its address (Section 3.2.1). The allowlist was not enabled in the test network, where every node was started by the team on one host.

Consensus. Among the connected nodes, only validators can propose and sign blocks, and the set of validators changes only when more than half of the existing validators vote for the change [19]. Every change to the record must therefore be committed by a quorum of validators, so no single organisation can change the record on its own. This layer is implemented, and it is what makes the network permissioned.

Accounts. Besu can also refuse transactions from accounts that are not on an accounts allowlist [24]. Voters are accounts, so such a list could in principle hold the voter roll, but the prototype does not use it. Instead, the voting contract holds the roll and checks every vote against it (Section 3.4.2). This has two advantages. An allowlist is kept separately on each node [24], whereas every change to the contract's roll is agreed by the validators and copied to every node. An allowlist can also only say whether an account may send transactions, whereas the contract can also check that the voter has not already voted.

The administrator. The administrator is the account that deployed the voting contract, and it is the only account that can register voters, open voting and close voting (Section 3.3). It has no function that changes a vote or the tally directly, and voters can only cast their own vote.

In the prototype, the administrator is also one of the validators. Like any other transaction, an administrator action takes effect only when a quorum of validators commits it. However, the validators check only that the transaction follows the contract's rules, not whether the administrator should have sent it. If the administrator registers an address that belongs to no real voter, the validators still accept it. The quorum therefore protects the record against a faulty validator, but not against misuse of the administrator key.

3.3 The Voting Contract

The voting contract, VotingSystem, holds all of the election's data. Besu runs it in the EVM. Every validator executes the transactions in each committed block, in the same order, so all validators arrive at the same contract state. Its internal logic is covered by P. Sheng [42].

Figure 5 shows the data the contract stores. The administrator's address, the number of candidates and whether voting is open are single values. The voter roll is a mapping, a lookup table from each address to whether it is registered, and a second mapping holds each voter's status. A third mapping holds the tally, with one counter per candidate.

```
enum VoterState { NotVoted, Processing, Voted }

address public admin;                // administrator account
uint8   public numCandidates;
bool    public votingOpen;

mapping(address => bool)       public registeredVoters; // roll
mapping(address => VoterState) public voterState;       // status
mapping(uint8   => uint256)    public voteCount;        // tally

event VoterRegistered(address indexed voter);
event VoteCast(address indexed voter, uint8 candidateId);
event VotingOpened();
event VotingClosed();
```

Figure 5: Stored data and events of the VotingSystem contract (excerpt; comments added).

The contract has six functions, in three groups:

Setting up the election. registerVoter() adds an address to the voter roll with the status NotVoted, and openVoting() and closeVoting() start and end the voting period. Only the administrator, the account that deployed the contract, can call these three functions.

Voting. castVote() is the only function a voter calls, and it validates each vote: a vote is accepted only if voting is open, the sender is registered and has not yet voted, and the candidate number is valid. It then adds one to that candidate's count, sets the voter's status to Voted, and records a VoteCast event. If any check fails, the call is reverted and nothing changes.

Reading. getTally() returns a candidate's current count, and getVoterState() returns a voter's status. These functions only read the state, so they do not create transactions, and any validator can answer them from its own copy.

An event is a record that a contract writes into a transaction's receipt. It is stored with the block and can be searched later, so it forms a permanent log of what the contract did. VotingSystem emits one event for each kind of action: VoterRegistered, VotingOpened, VotingClosed, and VoteCast (Figure 5). VoteCast records the address of the voter and the candidate chosen. Section 3.5 shows what one vote and its VoteCast event look like once stored.

3.4 Vote Validation

Validating a vote means checking that it is well-formed and allowed, and having the validators agree to add it to the record. This happens in three steps: a node accepts the signed vote into its transaction pool, the voting contract checks it against the voter roll, and the validators agree on the block that contains it.

3.4.1 Transactions and the Transaction Pool

In Besu, as on any Ethereum network, every change to the chain is made by a transaction: a message signed with the sender's private key, which no node can alter without invalidating

the signature. A vote is therefore a signed transaction that calls the voting contract. A node that receives a transaction checks its signature and format, places it in its transaction pool and passes it on to its peers [27].

The transaction pool is a waiting area for transactions that a node has received but that are not yet in a block. Each node keeps its own pool, and the validators do not agree on its contents as they do for blocks, so two nodes may hold slightly different waiting transactions at the same moment. A pool can also hold only a limited number of transactions.

Besu's sequenced pool, which is recommended for private networks, keeps transactions in order of arrival and, when it is full, evicts the newest transactions first [27]. A transaction in the pool is not yet part of the record: it joins the record only when it is included in a committed block.

3.4.2 Checking Eligibility Against the Voter Roll

The contract checks each vote against the voter roll: castVote() accepts a vote only if the sender is registered and has not yet voted (Section 3.3). A vote that fails this check is still included in a block, but it changes nothing: its receipt records it as failed, and it emits no VoteCast event (Section 3.5.2).

In the prototype, a voter is identified only by an address. Voters' keys were generated outside the system, so the link between a person and an address is assumed rather than verified. In a real election, this link would come from the identity service behind the production client (Section 3.1).

3.4.3 Consensus

Besu's documentation lists two proof-of-authority consensus protocols for private networks, QBFT and IBFT 2.0 [16]. Both have the properties described in Section 2.5.

We chose IBFT 2.0 because it has been formally proven both safe and live [20]. Honest validators never commit conflicting blocks, and the network keeps producing blocks even when messages between validators are delayed.

3.5 Storage

Each validator stores three kinds of data: the blocks, a receipt for every transaction, and the world state. A block's header holds a hash of its transactions, of their receipts, and of the world state after the block. These hashes, called roots, let a validator check that its copy of each store matches the copies held by the other validators.

3.5.1 Blocks

A block has a header and a body, as Figure A.1 in Appendix A shows for a block containing one vote. The body lists the block's transactions in the order they were executed. For a vote, the stored transaction holds the voter's address, the contract's address, the function called with the chosen candidate as its argument, and the voter's signature. The header records the block's position in the chain, links it to the block before it, names the validator that proposed it, carries the signatures of the validators that committed it, and holds the three roots. Blocks are never removed, so every vote stays in the chain for as long as the chain is kept.

A new block is produced every block period. The block period of 2 s and the round-change

timeout of 4 s follow Besu's IBFT 2.0 example configuration [33], in which the timeout is twice the block period, as Besu's documentation advises [19]. The values were not tuned for this workload. With a 2 s period, a vote waits at most about 2 s for the next block, which is short for a voter. A shorter period would reduce this wait but increase consensus traffic.

A block is also produced every period when there are no votes, which amounts to 43,200 blocks per day. Besu can delay empty blocks with an empty block period, but in version 24.12.0 this was an early-access feature for QBFT only [34], and users reported that it did not yet work reliably even there [35]. In our tests, setting the option with IBFT 2.0 had no effect, so the prototype produces empty blocks throughout, which adds to storage growth (Section 5). Later releases fixed these problems: Besu 26.5.0 made the empty block period a standard option for both IBFT 2.0 and QBFT [41], and the current documentation lists it for both [19], [26], so a newer version should remove most of this overhead.

3.5.2 Receipts and Events

When a validator executes a transaction, it produces a receipt recording whether the transaction succeeded, the gas it used and any events it emitted. Receipts are not sent between validators: each validator produces its own by executing the block, and the receipts root in the block header lets it confirm that its receipts match those of the others. Like blocks, receipts are never removed.

For a successful vote, the receipt records success and holds one VoteCast event with the voter's address and the candidate number (Figure A.2, Appendix A). The event is stored as topics and data: the first topic identifies the event as VoteCast, the second holds the voter's address, and the data holds the candidate. Because the voter's address is indexed as a topic, events can be searched by voter. For a vote that fails the contract's checks, such as a second vote from the same voter, the transaction is still stored in the block, but its receipt records failure and holds no event. Counting VoteCast events therefore counts exactly the votes that succeeded, which is what the audit in Section 3.6 relies on.

3.5.3 World State

The world state holds the current value of every account and every contract variable, including the voter roll, each voter's status and the tally. A vote adds no new record here. Instead, it changes two values in the contract's storage. For the vote in Figure A.1, the voter's status becomes Voted and candidate 2's count increases by one:

- voterState[0x7d1c. . .] NotVoted $\rightarrow$ Voted
- voteCount[2] 1,203 $\rightarrow$ 1,204

Besu offers two formats for storing the world state, Bonsai Tries and Forest of Tries [29]. The format affects only the world state; blocks and receipts are stored in the same way in both. The prototype uses Bonsai, and Figure 6 compares it with Forest.

Both formats organise the state as a trie, a tree whose root summarises the whole state in a single hash. In Forest, every trie node is stored under its own hash. Each block that changes the state adds new trie nodes and a new root, while the old ones stay in the database, so the store keeps every past version of the state and grows with every block [29]. Bonsai instead stores trie nodes by their position in the tree and keeps the current

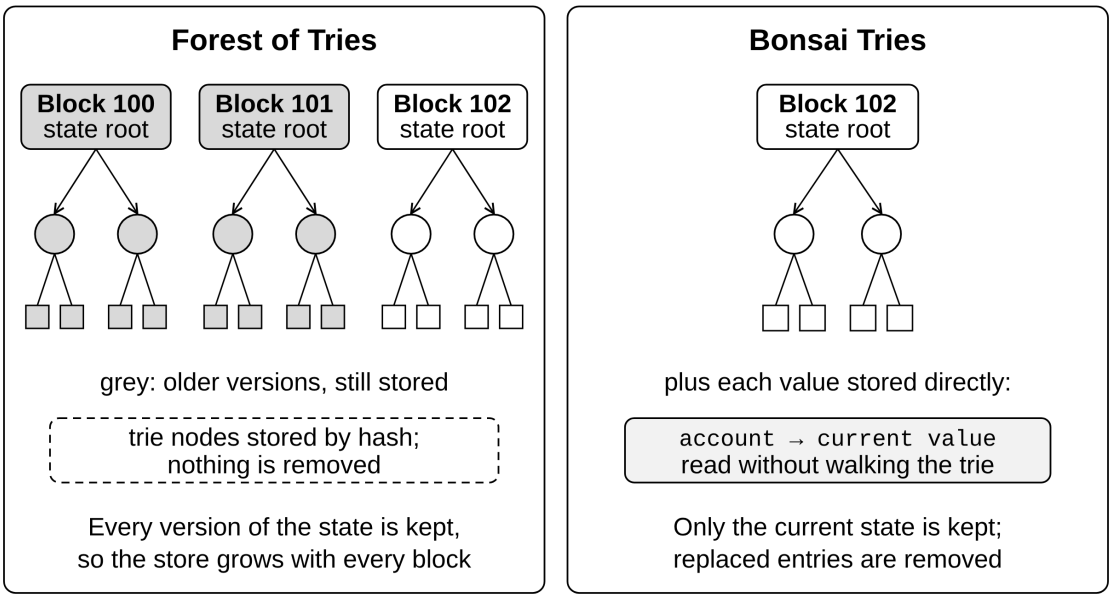


Figure 6: How the Forest and Bonsai formats store the world state across three blocks (simplified; in practice unchanged trie nodes are shared between versions).

value of each account directly, so a value can be read without walking the trie. Nodes that are replaced are removed, so the store holds essentially one version of the state [29]. By default, Bonsai can rebuild the state of only the last 512 blocks, about 17 minutes at a 2 s block period [29].

The difference matters for a national election, in which every vote changes the state. Forest suits nodes that must answer questions about any past block, but its database grows faster and its lookups become slower, and Besu does not recommend it for Ethereum Mainnet [29]. Bonsai keeps the store smaller and reads faster, at the cost of not answering questions about old states. The current tally and the full audit trail do not need old states, since both are available from the current state and the stored receipts (Section 3.5.2), so the prototype uses Bonsai.

3.6 Tallying

The tally is kept by the contract itself. Because castVote() updates the count directly, each vote is counted as soon as the block that contains it is committed. There is no separate counting step after voting closes: the tally is always current, and closing the election simply stops further votes. The status field also has a Processing value, but castVote() sets it and replaces it within the same transaction, so only NotVoted and Voted can be observed from outside.

The VoteCast events give the election an audit trail that is independent of the stored tally. Anyone with read access to a validator can collect every VoteCast event, recount the votes, and compare the result with getTally(). Each validator holds the full chain, so each can be audited from its own copy.

The same events expose a limitation. Because VoteCast records both the voter's address and the candidate, every vote can be linked to the address that cast it, and that link is copied to every validator. The prototype therefore provides no ballot secrecy, which is why Section 2.7 places it outside this evaluation.

4. Evaluation Method

The prototype was evaluated with two tests on a local Besu network, each built around one rule that a voting system must not break. The tally audit (Section 4.3) checks that every validator holds the same record and that the tally can be recounted independently, including after a validator crashes. The storage test (Section 4.4) measures how much storage the record of a national election needs.

4.1 Test Environment

All tests ran on a single computer, with every validator as a separate Besu process on the same virtual machine. This keeps the network identical between runs and allows a validator to be stopped and restarted on command, which the crash scenario in Section 4.3 requires. The machine was a VirtualBox virtual machine with 4 virtual processors and 8 GB of memory, without swap space, on a laptop with a 14-core Intel Core Ultra 5 125H processor and 32 GB of memory, running Ubuntu 24.04, Besu 24.12.0 and Java 21.

Each validator ran as its own system service with a fixed memory limit, so that a validator that ran out of memory would fail alone rather than stop the whole machine.

The tests used a harness that P. Sheng [42] built around Hyperledger Caliper, a blockchain benchmarking tool. Its custom connector signs each vote with the key of a different pre-registered voter, so that every test vote is a valid first vote, and it records the confirmation time of every vote. Scripts in the harness stop a validator without warning by killing its process, restart it, and monitor block height and memory use throughout. The two tests in Sections 4.3 and 4.4, and the scripts that analyse their results, were written and run for this report on that harness.

4.2 Election Load Scenarios

This evaluation uses two demand scenarios. The realistic scenario is the 2023 election-day average of about 25 votes per second (Section 2.1). The worst-case scenario, used in the capacity tests by P. Sheng [42], assumes that all of the 3,677,600 people enrolled to vote as at 30 September 2026 [45], about 3.7 million, vote on a single ten-hour day, which gives 3,700,000 / (10 $\times$ 3,600) $\approx$ 103, or about 100 votes per second. Both are averages over a whole day rather than peaks, because no hourly figures exist, so even the worst case does not describe the busiest hour of a real election.

The two scenarios serve different purposes. The capacity tests by P. Sheng [42] use the worst case, because capacity must cover the heaviest demand that could plausibly occur. The tests in this report use the realistic scenario of 25 votes per second as a sustained load, since what they check does not depend on the rate: storage depends on how many votes are stored rather than how fast they arrive, and a correct tally must be correct at any rate.

4.3 Tally Audit Across Validators

Every validator must hold the same vote record, and the tally must be checkable without relying on the contract's own counter. This test checks both, in normal operation and after a validator has crashed and caught up.

A 4-validator network received 10,000 votes at 25 per second. In the second scenario, validator 4 was stopped without warning during voting, held down for 60 s, restarted, and allowed to catch up before voting closed. Each scenario was run three times. After voting

closed, an audit script chose a common block height, the lowest latest block across all validators, so that every validator was checked at the same point. For each validator, it then:

read the block hash at that height, which must be identical on every validator;

read the tally for each candidate with getTally();

recounted the votes independently from the VoteCast events (Section 3.6); and

checked that the number of events equalled the number of distinct voters, so that no voter was counted twice.

The totals were also compared with the number of votes that the test client saw confirmed, and the time taken by each recount was recorded to estimate how long auditing a national election would take, which is judged against the one-day target in Section 2.1.

4.4 Storage Growth

Every validator must be able to store the full record of a national election. This test measures how storage grows with time, with registrations, and with votes.

A new 4-validator network was run in three phases: 30 minutes with no transactions, so that only empty blocks were produced; the registration of 10,000 voters; and 10,000 votes at 25 per second. The size of each validator's data directory was measured after each phase, following a 60 s pause to let background database maintenance settle. The storage used per empty block, per registration and per vote was then calculated, with the growth from empty blocks produced during each phase subtracted.

These rates were scaled up to estimate the storage each validator would need for an election in which all of about 3.7 million enrolled voters [45] register and vote, with a block produced every 2 s over 21 days. This scales the 10,000-vote sample up by about 370 times and assumes that storage grows in proportion to the number of transactions and blocks. The test was also run once, so the estimate is indicative.

4.5 Threats to Validity

Both tests ran on the same small setup, which limits what their results can show in four ways:

Shared hardware. All four validators and the test client shared four processors and 8 GB of memory. This does not change whether the tallies match or how many bytes each vote adds, but it does affect timing, so the recount and catch-up times would likely be shorter on dedicated servers.

No real network. Validators exchanged messages within one machine, so the delays of a network spread across New Zealand are absent. This mainly affects how quickly a crashed validator catches up, not the content of the record.

A small sample scaled up. Both national-scale estimates, of recount time and of storage, scale up results from 10,000 votes by about 370 times and assume that the cost grows in proportion to the number of votes and blocks.

A simple workload. Every test vote was a valid first vote from a different voter. A real election would also include failed attempts, such as second votes, which are stored in

blocks with their receipts but emit no VoteCast event (Section 3.5.2), so they would add storage that was not measured here.

5. Results

5.1 Tally Audit Across Validators

In each run, all four validators reported the same block hash at the audit height, getTally() on every validator returned the same totals as the recount from VoteCast events (3,334, 3,334 and 3,332 votes for the three candidates), the 10,000 events came from 10,000 different voters, and the total equalled the 10,000 votes that the client saw confirmed. This was also true of validator 4 in the second scenario, which had crashed and caught up during voting. The audit output also showed that every VoteCast event records the voter's address together with the chosen candidate.

Figure 7(a) shows the time taken to recount 10,000 votes from the VoteCast events. Averaged over the four validators, a run took between 1.87 and 2.03 s, and individual validators took between 1.72 and 2.21 s. Scaled in proportion to the number of votes, the run averages give 691 to 751 s, about 11.5 to 12.5 minutes, for one validator of this machine to recount 3.7 million votes, and 933 to 1,015 s, about 16 to 17 minutes, for 5 million, the election size used in the 2025 MACI reports (Figure 7(b)). These estimates assume an election like the test, in which almost every transaction is a successful vote.

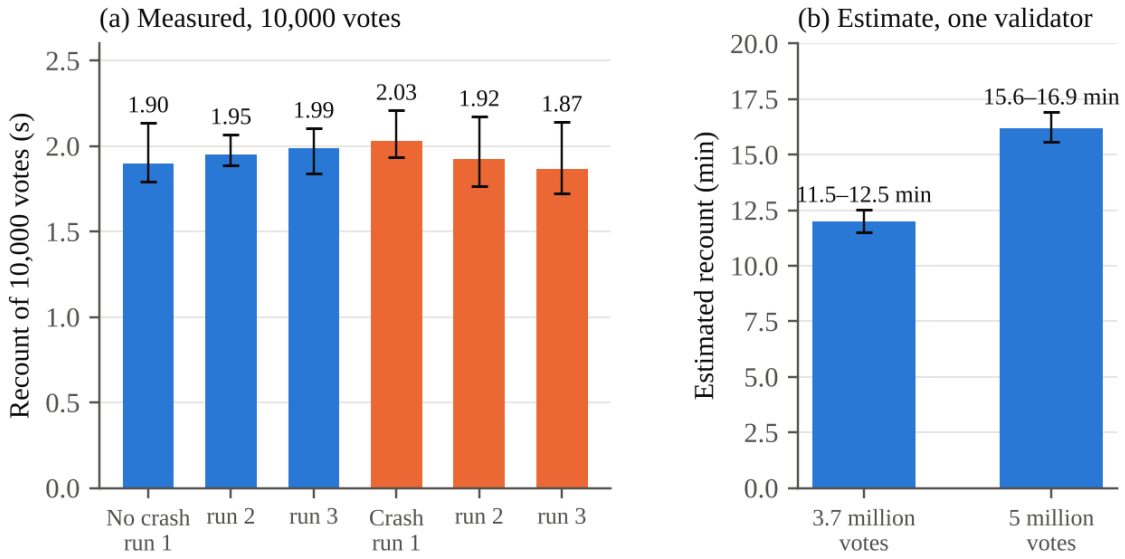


Figure 7: Time for one validator to recount the votes from VoteCast events, measured for 10,000 votes in each run (a; bars show the mean of the four validators and error bars the fastest and slowest) and scaled to a national election (b).

During the crash runs, the three remaining validators kept producing blocks while validator 4 was down, adding 22 blocks in the 60 s hold rather than the 30 that a 2 s block period would give. Validator 4 caught up 7.7 to 8.5 s after it was restarted, and its block hash then matched the others. All 10,000 votes were confirmed in every run of both scenarios. The crash barely changed the median confirmation time. Confirmation latency is discussed further by P. Sheng [42].

5.2 Storage Growth

Figure 8 shows the size of one validator's data directory at each measurement. The four validators never differed by more than 200 bytes. The new chain used 0.41 MB. After 30 minutes and 931 empty blocks it used 2.24 MB, after 10,000 registrations 15.5 MB, and after 10,000 votes 50.9 MB. During the two 60 s pauses, when only empty blocks were produced, the directory grew by about 1,880 and 1,820 bytes per block, close to the figure from the idle phase even though the chain by then held 10,000 registered voters.

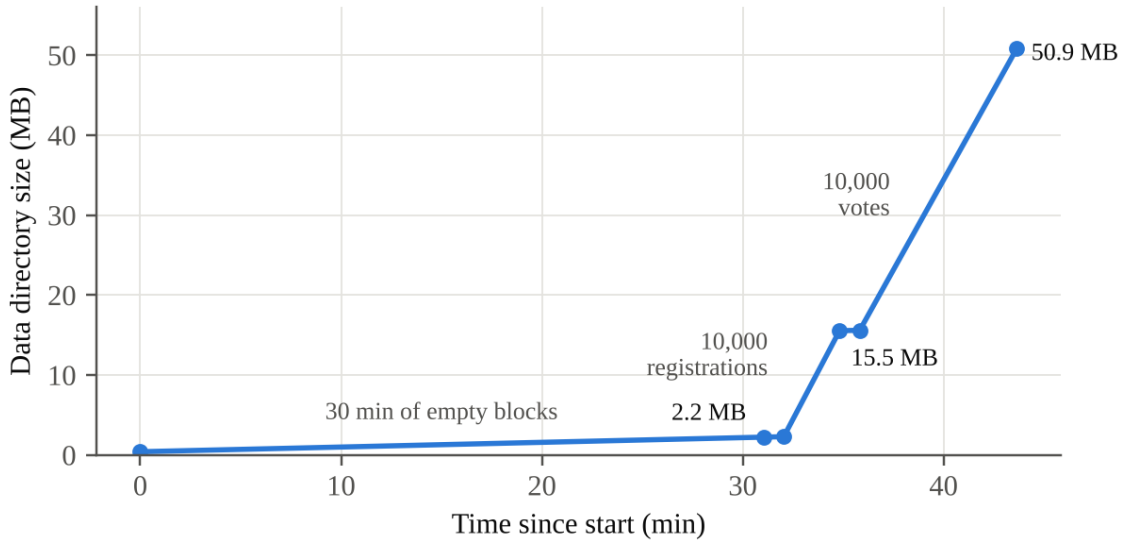


Figure 8: Measured growth of one validator's data directory during the storage test.

In the sample, votes accounted for 34.8 MB of the growth (3,480 bytes each), registrations 13.1 MB (1,309 bytes each), and empty blocks 2.6 MB (1,961 bytes each) (Figure 9(a)). Scaled to 3.7 million registrations, 3.7 million votes and 907,200 blocks over 21 days, these rates give about 19.5 GB per validator, of which votes account for 12.9 GB, registrations 4.8 GB and empty blocks 1.8 GB (Figure 9(b)).

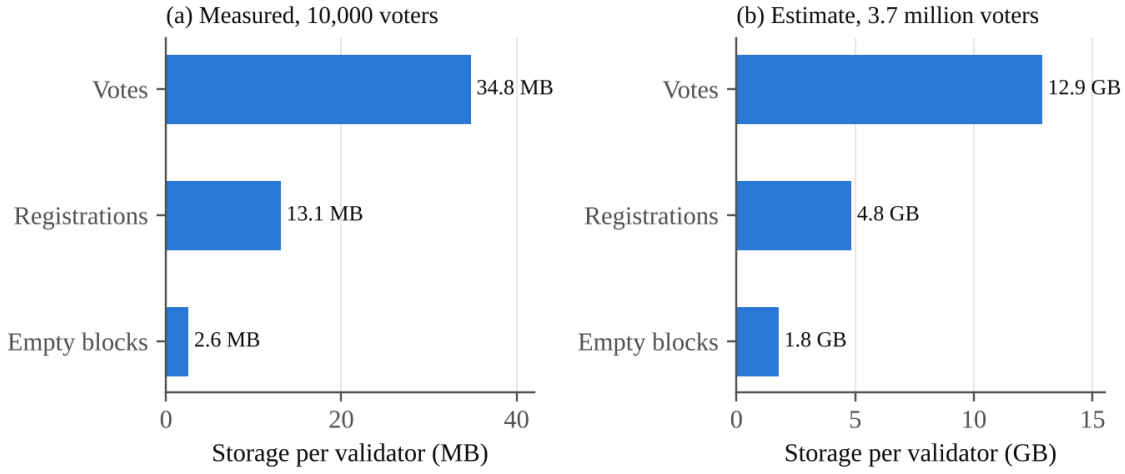


Figure 9: Storage per validator by type of record, measured for 10,000 voters (a) and estimated for 3.7 million voters over 21 days (b).

6. Discussion

The aim of this section is to separate what the prototype demonstrates from what remains unproven, and to distinguish limitations of this implementation, limitations of the current design, and challenges that any form of online voting would face.

6.1 Tally Integrity and Audit

6.1.1 Consistency Across Validators

The tally audit passed in every run. The validator that crashed and caught up during voting held exactly the same record as the others. This is the property a private network is meant to provide: the result does not depend on trusting the contract's counter or any one validator, because each organisation that runs a validator can recount the votes from its own copy of the chain and compare its answer with the others. However, this openness extends only to those with read access. The public could check the record only if the operators published it or gave observers read access, much as scrutineers observe the paper count today [1].

6.1.2 Recount Time at National Scale

A full recount of 3.7 million votes was estimated to take 11.5 to 12.5 minutes per validator on the test machine. This assumes that recount time grows in proportion to the number of votes, which is reasonable because the recount reads the events in fixed-size ranges of blocks and does the same work for each event. It also assumes an election like the test, in which almost every transaction is a successful vote. Failed attempts add transactions to the block but no events in the receipt (Section 3.5.2), so they would add little to the recount. Section 2.1 set two targets for an online system: a tally within one hour of voting closing, and evidence that the tally is correct within one day [4]. On these estimates, the prototype would meet both with a wide margin. Its tally is available as soon as voting closes, because the contract keeps it current (Section 3.6), and even if a national recount took three times longer than estimated, each organisation could check the result within about 40 minutes, well before the preliminary results that are normally released two to four hours after voting closes [4].

6.1.3 What the Audit Cannot Catch

The audit checks that the record is consistent, not that what was recorded was legitimate. Ballot integrity is the principle of one person, one vote (Section 2.1): only eligible voters may vote, each may vote only once, and every vote cast must be counted. The chain enforces the last two, as the audit showed, but the first depends on two parts of the system that lie outside it.

The first is the administrator key. Only the administrator can add voters to the roll, and the validators accept any registration it sends, because they only check that a transaction follows the contract's rules (Section 3.2.3). Whoever holds the key could therefore register addresses that belong to no real voter and cast votes from them. Those votes would be counted, re-counted, and audited exactly like real ones, so the audit would not detect them. The risk comes from the key being held in one place: in the prototype, it is a single key that also belongs to a validator. It could be reduced by splitting the key with multi-party computation, in which each of several organisations holds only a share of the key and a transaction can be signed only when enough of them take part [40]. No single organisation

could then change the voter roll alone. Because every registration is also recorded on the chain, auditors could compare the number of registered addresses with the official electoral roll.

The second gap is the link between an address and a person. The contract knows voters only as addresses, and in the prototype the link between an address and a real voter was assumed rather than checked (Section 3.4.2). In a deployment, this link would be made when the voter signs in, for example through RealMe (Section 3.1), and the administrator would register the voter's address only after that check. The chain can stop any registered address from voting twice, but it cannot tell whether two addresses belong to the same person; that has to be checked when voters are registered. If the identity check were weak, or a voter's key were stolen or sold, the chain would accept the vote as valid. Ballot integrity therefore depends as much on the identity service, and on how voters' keys are created and protected, as on the blockchain, and these lie outside what this project tested. Neither gap is specific to blockchains, since any online voting system must control who registers voters and how voters prove who they are. Both are limitations of the current design that known techniques could address, but neither was implemented or tested in this project.

6.2 Storage at National Scale

On this evidence, storage is unlikely to be a barrier. About 19.5 GB per validator, with every enrolled voter registered and voting, would fit on any current server. Because every validator holds the full record, a network of n validators stores n copies, which is still small.

The estimate scales a 10,000-voter sample up about 370 times and assumes that storage grows in proportion to the number of blocks, registrations and votes. There are reasons to expect this. Each vote adds the same records: one transaction, one receipt with one event, and a change to two stored values (Section 3.5). The cost of an empty block also stayed at about 1.8 to 1.9 kB after 10,000 voters had registered, so it did not grow with the size of the voter roll. Growth could still become faster than linear if the tree that stores the world state became deeper as voters were added.

The estimate also assumes that every transaction is a successful first vote, which the test did not challenge. Because gas is free, nothing on the chain stops a registered voter, or anyone who obtains a voter's key, from sending further votes. Each one fails the contract's checks but is still stored in a block with its receipt (Section 3.5.2), so it costs every validator storage without ever being counted. The size of a failed vote was not measured. It should be smaller than the 3,480 bytes of a successful vote, since it changes no stored values and emits no event, but it is not zero. As an upper bound, if a failed attempt cost as much as a successful vote, a single key sending 100 transactions per second for the ten hours of election day would add about 12.5 GB to every validator, nearly doubling the estimate for the whole election. Storage therefore depends not only on how many people vote but on how many transactions the network accepts. The voting platform should check getVoterState() before sending a vote and limit how often each address can submit, so that the contract's check is the last line of defence rather than the only one.

Empty blocks are the part of the storage that does not depend on voting. They add about 85 MB per validator per day, or 1.8 GB over the 21-day election period. The empty block period described in Section 3.5.1 would remove most of this, but it was not available for IBFT 2.0 in the Besu version used. This was a limitation of that Besu version rather than

of IBFT 2.0. Since Besu 26.5.0, the empty block period has been a standard option for both IBFT 2.0 and QBFT [19], [26], [41], so upgrading Besu, rather than changing the consensus protocol, should remove most of this overhead, although this would need to be tested.

6.3 Capacity and Availability

Capacity and availability were measured by P. Sheng [42], who recommends a network of 10 to 13 validators. These sizes tolerate three or four failed validators while a typical vote is still confirmed in about 2 s. Consensus itself took only about 0.2 s per block even with thirteen validators. With the tolerated number of validators offline, the network kept producing blocks and lost no votes, although the slowest votes waited up to about 60 s for a round change [42].

This choice balances the two requirements. More validators tolerate more failures, but every block then needs agreement from more of them, which lowers capacity (Section 2.5). The open question is capacity. The worst case is an average of about 100 votes per second on election day (Section 4.2). On the shared test machine, 10 and 13 validators sustained only about 25 and 10 votes per second. That limit came from processor time and memory, not consensus, so whether these sizes meet the target on separate machines remains to be measured. If more validators fail than the network tolerates, it stops without creating conflicting records. A larger transaction pool, such as 65,536 instead of Besu's default 4,096, keeps votes from being lost while it recovers [42].

6.4 Voter Privacy and Ballot Secrecy

Ballot secrecy is the clearest requirement that the prototype fails. Section 2.1 defines it as no record linking a voter's identity to their choice [7], yet every vote is stored with exactly that link. The VoteCast event records the voter's address and the candidate, and the transaction itself contains the candidate number in plain form (Section 3.5.1). The link is copied to every validator and, because committed blocks cannot be changed, it can never be deleted. A private network limits who can read the chain, but every validator operator, and anyone they give read access to, can see how each address voted, and the administrator who registered each address can connect it to a person. Each of these parties is therefore a point at which the link between voters and their choices could leak, and a single leak would expose those votes permanently.

This is the tension described in Section 2.1 in its sharpest form. The audit in Section 6.1 works precisely because each vote is stored openly, so anyone with read access can recount every event. Adding secrecy means storing votes in a form that hides each choice while still allowing the total to be checked. One approach is to encrypt each vote so that only a key shared among several organisations can reveal the result, and only after voting closes. Another is to prove that the tally is correct with zero-knowledge proofs, as MACI does [4], [5]. Both move work from the infrastructure into cryptography, and Section 6.5 shows what this cost MACI in time. These possible fixes show that the lack of secrecy comes from how the prototype stores votes, not from private blockchains themselves. Until votes can be hidden while still allowing a recount, the prototype could not be used in a real election.

Voter privacy, the ability to mark and cast a ballot alone and without help [7], is a different problem. It depends on the voter's device and surroundings, which no server-side design can control, and it is a main reason that remote voting is considered risky [13], [14]. A

voter at home may be watched or pressured by someone standing next to them, and nothing in the prototype can detect or prevent this. MACI addresses part of this problem by letting voters secretly change the key their vote depends on, so that a vote cast under pressure can later be overridden [4], [5]. The prototype has no such mechanism, so it provides neither voter privacy nor ballot secrecy. Unlike secrecy, voter privacy is a challenge for remote voting in general, and it would remain whatever infrastructure recorded the votes.

6.5 Comparison with MACI

Compared with MACI on public networks, the private network was much faster. Kwon [5] measured average confirmation times of 10.7 s on Sepolia, 4.6 s on Gnosis Chiado and 1.3 s on Optimism Sepolia at 2.5 votes per second, while in the tests by P. Sheng [42] the private network confirmed votes in an average of 1.47 s at 10 votes per second on four validators, with no transaction fees. Browne [4] estimated that generating the proofs for a MACI election of five million votes would take weeks, while an independent recount of the same number of votes here would take about 16 to 17 minutes (Section 5.1).

The speed comes mainly from the design. On a public network, every vote competes with other users' transactions for space in blocks, and agreement relies on mechanisms built for participants who do not trust each other. Here, a few known validators agree by exchanging signed messages, a block is produced every 2 s, and only the election uses the chain (Section 2.4). MACI is also slower because it encrypts every vote and proves the tally with zero-knowledge proofs. The two systems therefore make different trade-offs: MACI gives ballot secrecy and resistance to coercion, while the prototype gives speed, a fast recount and no fees, partly because it stores every vote in plain view. Neither is suitable on its own. A system for New Zealand would need the throughput and auditability of a private network together with a way of keeping votes secret that does not bring back MACI's tallying cost.

A conventional system, in which the election authority stores votes in its own database, would be simpler and faster still, since it needs no agreement between validators. Estonia's internet voting is centralised in this way [13]. What the private network adds is that no single organisation holds the only copy of the record. Every change must be agreed by a quorum of validators run by different organisations. Each validator holds an identical copy, and it can recount the result independently (Section 6.1), so rewriting the record would require a quorum of those organisations to collude. What it does not add is protection for voters' devices, ballot secrecy or a check on the administrator, which is why Park et al. [14] argue that a blockchain does not by itself solve the problems of internet voting. A private network is therefore best understood as a way of sharing trust in the record among several known organisations, not as a way of removing the need for trust.

6.6 Practical Deployment

A real deployment would place each validator with a different trusted organisation, such as the Electoral Commission, an independent auditor and a registrar, at separate sites, with the node allowlist enabled so that only these validators can connect (Section 3.2.3). Storage would not constrain the hardware, since about 20 GB per validator is enough (Section 6.2), and the processor and memory needs follow from the capacity results of P. Sheng [42]. Each validator would also need monitoring, since Besu 24.12.0 occasionally stopped taking part in consensus under load until it was restarted [42]. The number of validators is a governance decision as much as a technical one: more organisations spread trust more widely and tolerate more failures, but slow consensus (Section 6.3). The measured results

point to 10 to 13 validators as a balance between the two [42].

Keys need more care than the prototype gave them. The administrator key should be separated from the validators and split between several organisations, as Section 6.1 describes. Voters also need keys, and these must be tied to real people through an identity service such as RealMe (Section 3.1), with a way to recover if a device or key is lost.

Running costs move from fees to servers. On a public network every vote pays a transaction fee, whereas on this network gas is free and the cost lies in operating the validators. The same network could be reused for later elections by deploying a new voting contract. Keeping the same network would let the organisations reuse their validators and permissions, and the records of earlier elections would stay on the chain for later audit. The cost of keeping it running between elections depends on whether empty blocks can be avoided (Section 6.2).

Online voting would not replace paper voting in New Zealand, since some voters will always need or prefer to vote in person, so a hybrid system, in which online and paper voting run side by side, is worth investigating. Ballot integrity then requires that no voter can vote both online and on paper. Because the voter roll is held in the contract, voting places could check it and mark a voter off on the chain when they issue a paper ballot, so that a later online vote from the same voter would be rejected and an online voter would be refused a paper ballot. This would need a small addition to the contract, a function that lets voting places mark a voter as having voted, and it would need voting places to be connected to the network. Where they were not, paper votes from voters who also appeared to have voted online could be checked before counting, as special votes are today [8]. The paper votes themselves would still be counted by hand, so the chain would hold the online votes and the voting status of every voter, not every vote. None of the measures in this section was built or tested in this project. They indicate the design work that a deployment would need, not capabilities that the prototype has demonstrated.

6.7 Suitability Against the Criteria

Table 1 summarises the prototype against the six criteria of Section 2.1, together with the capacity, scale and tally time needed for a national election. The verdicts describe what this evaluation demonstrated under the test conditions in Section 4.5, not what has been proven for a national election.

Table 1: Suitability of the prototype against the criteria of Section 2.1, with capacity, scale and tally time.

Criterion	Verdict	Evidence (section)
Transparency	Partly met	Every transaction, event and the contract can be inspected by anyone with read access, but the public sees the record only if the operators publish it or give observers access (6.1).
Voter privacy	Not met	Depends on the voter's device and surroundings, which the system cannot control (6.4).
Ballot secrecy	Not met	Every vote is stored with the voter's address and the candidate chosen, and whoever registered the address can link it to a person. This link is held by every validator and cannot be deleted (6.4).

Criterion	Verdict	Evidence (section)
Auditability	Met	The stored tally matched an independent recount on every validator in every run (6.1).
Ballot integrity	Partly met	One vote per registered address is enforced and audited, but the voter roll depends on a single administrator key and an identity link outside the system (6.1).
Availability	Partly met	Voting continues through validator failures up to a limit set by the number of validators. Beyond it, blocks stop, and accepted votes can be lost if an outage is long (6.3).
Capacity	Partly met	Four validators sustained about 150 votes per second, above the 100 votes per second target, but the recommended 10 to 13 validators were not shown to reach it on the shared test machine [42] (6.3).
Scale	Met	Storage: About 19.5 GB per validator for 3.7 million voters (6.2). Tally: about 12 minutes for a national recount if almost all votes are successful, within the one-day target (6.1).

Taken together, the findings support a qualified position. Within the limits of the tests, the prototype demonstrates that a private, permissioned blockchain can provide properties that are hard to obtain from a conventional central server: one consistent record that several organisations can each hold and check, a tally that any of them can recount in minutes, and a national record that fits in about 20 GB per validator. These are meaningful results for the auditability and integrity of an election record. They were shown, however, with 10,000 votes on four validators sharing one machine, and they do not make the prototype a complete voting system.

Three requirements are met only in part or not at all, and they differ in kind. Ballot secrecy fails because of the current design, which stores votes openly, and could be addressed with cryptographic techniques at some cost in performance. Ballot integrity depends on a single administrator key and an identity check outside the chain, which need careful design of registration and key management. Voter privacy is a challenge for remote voting in general rather than for blockchains. A private blockchain is therefore a promising foundation for the record-keeping part of online voting, but not a complete solution, and its suitability for a New Zealand general election depends on how these problems are addressed and on further testing at realistic scale.

7. Conclusions and Future Work

A New Zealand general election places demanding requirements on any online voting system. Earlier work found that a public blockchain approach, MACI, could meet the privacy requirements but was too slow to confirm votes and to prove a national tally [4], [5]. This project therefore investigated the alternative that work pointed to: a private, permissioned blockchain built on Hyperledger Besu with IBFT 2.0, in which a known group of organisations shares the election record. This report evaluated the network's infrastructure, testing whether every validator keeps the same, independently checkable tally, including after a validator failure, and how much storage a national election would need, while drawing on the companion report by P. Sheng [42] for capacity and availability.

The tests show that the framework can provide part of what these requirements ask for. In every run, all validators held an identical record, including one that had crashed and recovered, and the stored tally matched an independent recount of the vote events. Scaled to a national election, a recount would take about 12 minutes per validator and the record would need about 20 GB on each, well within the one-day target for checking a tally and the capacity of ordinary servers. These results suggest that a private network can give several organisations a shared, auditable record without any one of them having to be trusted, which a conventional central system cannot offer. They do not show that the framework is ready for a national election. The results come from 10,000 votes on four validators sharing one machine, the national estimates assume that costs grow in proportion to the number of votes, and the prototype stores every vote openly, so it provides no ballot secrecy. Ballot integrity still relies on a single administrator key and an identity check outside the chain, and voter privacy depends on voters' devices and surroundings, which no server-side design can control. Some of these problems are limitations of the current design that could be addressed, while others, such as voter privacy, are challenges for any form of remote voting.

The most important future work follows from these gaps. Votes need to be stored in a form that hides each choice while still allowing an independent recount, for example by encrypting them under a key shared by several organisations, and the cost of doing so needs to be measured. The administrator key should be split between organisations, and registration should be linked to a verified identity so that each person receives exactly one address. The network should then be tested at realistic scale, with validators on separate machines at separate sites, a newer Besu version that supports the empty block period, and a workload that includes failed and repeated votes. Together with the capacity results of P. Sheng [42], this would show whether a single configuration can meet the requirements for capacity, availability and integrity at the same time. Overall, a private, permissioned blockchain is a potentially viable foundation for online voting in New Zealand, particularly for keeping a shared and auditable election record, but it is not a complete solution on its own. Its suitability depends on how the surrounding system is designed, and on whether further evaluation shows that it can meet the requirements it does not yet satisfy.

References

[1] Electoral Commission, "Election integrity." Elections NZ. Accessed: Oct. 14, 2026. [Online]. Available: https://elections.nz/democracy-in-nz/about-elections/election-integrity/

[2] Electoral Commission, "2023 General Election: Official result statistics," 2024. Accessed: Oct. 14, 2026. [Online]. Available: https://archive.electionresults.govt.nz/electionresults_2023/statistics/index.html

[3] J. Molineaux, "A working paper on online voting (internet voting, eVoting) in New Zealand," Auckland Univ. of Technology, Auckland, New Zealand, Draft working paper, Dec. 2018. [Online]. Available: https://thepolicyobservatory.aut.ac.nz/__data/assets/pdf_file/0006/233088/A-working-paper-on-internet-voting-in-New-Zealand-December-2018.pdf

[4] J. Browne, "Evaluating the scalability and suitability of MACI as an online solution for New Zealand's general election," Part IV project report, Dept. Elect., Comput., and Softw. Eng., Univ. of Auckland, Auckland, New Zealand, 2025.

[5] C. Kwon, "Evaluating the scalability and suitability of MACI as an online solution for New Zealand's general election," Part IV project report, Dept. Elect., Comput., and Softw. Eng., Univ. of Auckland, Auckland, New Zealand, 2025.

[6] M. Loncke and J. Dumortier, "Online voting: A legal perspective," *Int. Rev. Law, Comput. Technol.*, vol. 18, no. 1, pp. 59–79, 2004, doi: 10.1080/13600860410001674742.

[7] U.S. Election Assistance Commission, "Requirements for the Voluntary Voting System Guidelines 2.0," Feb. 2021. [Online]. Available: https://www.eac.gov/sites/default/files/TestingCertification/Voluntary_Voting_System_Guidelines_Version_2_0.pdf

[8] Electoral Commission, "Official results for the 2023 General Election," Media release, Nov. 3, 2023. [Online]. Available: https://elections.nz/media-and-news/2023/official-results-for-the-2023-general-election/

[9] Electoral Commission, "Last chance to vote," Media release, Oct. 13, 2023. [Online]. Available: https://elections.nz/media-and-news/2023/last-chance-to-vote

[10] Electoral Commission, "2023 General Election voting statistics." Elections NZ. Accessed: Oct. 14, 2026. [Online]. Available: https://elections.nz/democracy-in-nz/historical-events/2023-general-election/voting-statistics/

[11] State Electoral Office of Estonia, "Voter turnout at Riigikogu elections was 63.7%," Elections in Estonia, Mar. 8, 2023. Accessed: Oct. 16, 2026. [Online]. Available: https://www.valimised.ee/en/voter-turnout-riigikogu-elections-was-637

[12] P. Ehin, M. Solvak, J. Willemson, and P. Vinkel, "Internet voting in Estonia 2005–2019: Evidence from eleven elections," *Gov. Inf. Quart.*, vol. 39, no. 3, Art. no. 101718, 2022, doi: 10.1016/j.giq.2022.101718.

[13] D. Springall *et al.*, "Security analysis of the Estonian Internet voting system," in *Proc. 2014 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Scottsdale, AZ, USA, 2014, pp. 703–715, doi: 10.1145/2660267.2660315.

[14] S. Park, M. Specter, N. Narula, and R. L. Rivest, "Going from bad to worse: From Internet voting to blockchain voting," *J. Cybersecurity*, vol. 7, no. 1, Art. no. tyaa025, 2021, doi: 10.1093/cybsec/tyaa025.

[15] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[16] Hyperledger Besu, “Proof of authority consensus,” Besu documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://besu.hyperledger.org/private-networks/concepts/poa

[17] E. Androulaki *et al.*, “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in *Proc. 13th EuroSys Conf.*, Porto, Portugal, 2018, Art. no. 30, pp. 1–15, doi: 10.1145/3190508.3190538.

[18] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *Proc. 3rd Symp. Operating Syst. Design Implementation (OSDI)*, New Orleans, LA, USA, 1999, pp. 173–186.

[19] Hyperledger Besu, “IBFT 2.0,” Besu documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.besu-eth.org/private-networks/how-to/configure/consensus/ibft

[20] R. Saltini and D. Hyland-Wood, “IBFT 2.0: A safe and live variation of the IBFT blockchain consensus protocol for eventually synchronous networks,” 2019, *arXiv:1909.10194*.

[21] Hyperledger Besu, “Consensus protocols,” Besu documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://besu.hyperledger.org/private-networks/how-to/configure/consensus

[22] S. Pongnumkul, C. Siripanpornchana, and S. Thajchayapong, “Performance analysis of private blockchain platforms in varying workloads,” in *Proc. 26th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2017, pp. 1–6, doi: 10.1109/ICCCN.2017.8038517.

[23] L. Mostarda, A. Pinna, D. Sestili, and R. Tonelli, “Performance analysis of a Besu permissioned blockchain,” in *Advanced Information Networking and Applications* (Lecture Notes in Networks and Systems, vol. 655). Cham, Switzerland: Springer, 2023, pp. 279–291, doi: 10.1007/978-3-031-28694-0_26.

[24] Hyperledger Besu, “Permissioning,” Besu documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.besu-eth.org/private-networks/concepts/permissioning

[25] Hyperledger Besu, “enterprise-private.toml,” Besu 24.12.0 source code, GitHub. Accessed: Oct. 14, 2026. [Online]. Available: https://github.com/hyperledger/besu/blob/24.12.0/config/src/main/resources/profiles/enterprise-private.toml

[26] Hyperledger Besu, “QBFT,” Besu documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.besu-eth.org/private-networks/how-to/configure/consensus/qbft

[27] Hyperledger Besu, “Transaction pool,” Besu documentation. Accessed: Oct. 14, 2026. [Online]. Available: https://docs.besu-eth.org/public-networks/concepts/transactions/pool

[28] Hyperledger Besu, “Node keys,” Besu documentation. Accessed: Oct. 15, 2026. [Online]. Available: https://docs.besu-eth.org/public-networks/concepts/node-keys

[29] Hyperledger Besu, “Data storage formats,” Besu documentation. Accessed: Oct. 15, 2026. [Online]. Available: https://docs.besu-eth.org/public-networks/concepts/data-storage-formats

[30] Hyperledger Besu, “Welcome,” Besu documentation. Accessed: Oct. 15, 2026.

[Online]. Available: https://docs.besu-eth.org/

[31] Ethereum Foundation, "Node Discovery Protocol v4," devp2p specifications, GitHub. Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/ethereum/devp2p/blob/master/discv4.md

[32] Ethereum Foundation, "The RLPx transport protocol," devp2p specifications, GitHub. Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/ethereum/devp2p/blob/master/rlpx.md

[33] Hyperledger Besu, "Create a private network using IBFT 2.0," Besu documentation. Accessed: Oct. 16, 2026. [Online]. Available: https://docs.besu-eth.org/private-networks/tutorials/ibft

[34] Hyperledger Besu, "Release 24.10.0," besu-eth/besu, GitHub. Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/besu-eth/besu/releases/tag/24.10.0

[35] "QBFT no empty block," Issue #3810, besu-eth/besu, GitHub. Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/besu-eth/besu/issues/3810

[36] D. Jefferson, A. D. Rubin, B. Simons, and D. Wagner, "Analyzing internet voting security," Commun. ACM, vol. 47, no. 10, pp. 59–64, Oct. 2004.

[37] C. Culnane, M. Eldridge, A. Essex, and V. Teague, "Trust implications of DDoS protection in online elections," in Electronic Voting (E-Vote-ID 2017), Lecture Notes in Computer Science, vol. 10615. Cham, Switzerland: Springer, 2017, doi: 10.1007/978-3-319-68687-5_8.

[38] State Electoral Office of Estonia, "Statistics," Elections in Estonia. Accessed: Oct. 16, 2026. [Online]. Available: https://www.valimised.ee/en/archive/statistics

[39] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," white paper, 2014. Accessed: Oct. 16, 2026. [Online]. Available: https://ethereum.org/en/whitepaper/

[40] R. Gennaro and S. Goldfeder, "Fast multiparty threshold ECDSA with fast trustless setup," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Toronto, ON, Canada, 2018, pp. 1179–1194, doi: 10.1145/3243734.3243859.

[41] Hyperledger Besu, "Release 26.5.0," besu-eth/besu, GitHub. Accessed: Oct. 17, 2026. [Online]. Available: https://github.com/besu-eth/besu/releases/tag/26.5.0

[42] P. Sheng, "Assessing the Suitability of Blockchain Frameworks for New Zealand General Elections," 2026, Unpublished project report, University of Auckland.

[43] *Electoral Act 1993*, s. 167, New Zealand. Accessed: Oct. 18, 2026. [Online]. Available: https://www.nzlii.org/nz/legis/consol_act/ea1993103/s167.html

[44] R.-V. Tkachuk, D. Ilie, R. Robert, V. Kebande, and K. Tutschku, "On the performance and scalability of consensus mechanisms in privacy-enabled decentralized renewable energy marketplace," *Ann. Telecommun.*, vol. 79, no. 3–4, pp. 271–288, 2024, doi: 10.1007/s12243-023-00973-8.

[45] Electoral Commission, "Enrolment by general electorate," Elections NZ, 2026. Accessed: Oct. 18, 2026. [Online]. Available: https://elections.nz/stats-and-research/enrolment-statistics/enrolment-by-general-electorate

Appendix A Example Block and Receipt

This appendix shows what one successful vote looks like in storage, as described in Sections 3.5.1 and 3.5.2. Figure A.1 shows the block that contains the vote, and Figure A.2 shows the receipt that each validator produces when it executes that vote. Hashes and addresses are shortened, and the values are examples.

```
Block 500
  Header
    number             500            (position in the chain)
    parent hash        0x4a7b…        (links to block 499)
    proposer           0x9e2d…        (validator that proposed it)
    commit seals       …              (signatures of the quorum)
    transactions root  0x51c0…        (fingerprint of transactions)
    receipts root      0xb83f…        (fingerprint of receipts)
    state root         0x2d19…        (fingerprint of world state)
  Body: transactions
    from               0x7d1c…        (the voter)
    to                 0x42e5…        (VotingSystem contract)
    input              0x94a13541…02  (castVote(), candidate 2)
    signature          v, r, s        (the voter's signature)
```

Figure A.1: A block containing one vote, showing its header and the vote transaction (example values; hashes and addresses shortened).

```
Receipt of transaction 0x3f2a…
  status       1            (success; 0 means failed)
  gas used     …            (not charged on this network)
  Logs: one event
    emitted by 0x42e5…      (VotingSystem contract)
    topic 0    0xefffa0c0…  (identifies VoteCast)
    topic 1    0x…7d1c…     (the voter; indexed)
    data       0x…02        (candidateId = 2)
```

Figure A.2: The receipt of the vote in Figure A.1, holding its VoteCast event (example values; hashes and addresses shortened).